Quick Sort

Sorting Algorithms: Merge Sort vs. Quick Sort

1. Problem Description

Sorting is an important operation in computer science. It is used to arrange data in a specific order, such as ascending or descending order. Sorting makes data easier to search, organize, and process.

Quick Sort is a sorting algorithm that uses the Divide and Conquer approach to sort elements efficiently. It selects a pivot element and partitions the remaining elements into two groups based on their values. Then, it recursively sorts the two groups until the entire array is sorted.

2. How Quick Sort Works

Quick Sort follows four main steps:

1. **Base Case:** If the array contains zero or one element, it is already sorted.
2. **Pivot Selection:** Choose an element from the array as the pivot.
3. **Partitioning:** Divide the remaining elements into two groups based on their values compared to the pivot.
4. **Recursion:** Apply Quick Sort recursively to both groups until the entire array is sorted.

3. Pivot Selection and Partitioning

3.1 Pivot Selection Methods

The pivot is an element selected from the array to help divide the elements during the sorting process. Quick Sort uses different methods to select the pivot:

1. **First Element:** Selects the first element in the array.
2. **Last Element:** Selects the last element in the array.
3. **Middle Element:** Selects the middle element in the array.
4. **Random Element:** Selects a random element from the array.
5. **Median-of-Three:** Selects the first, middle, and last elements, then chooses the median value among them as the pivot.

3.2 Selected Pivot Method

In our Java implementation, we use the Median-of-Three method. It selects the median value from the first, middle, and last elements. This method helps produce more balanced partitions and can improve Quick Sort performance.

3.3 Partitioning

After selecting the pivot, Quick Sort divides the remaining elements into two groups:

- **S1:** Contains elements smaller than or equal to the pivot (≤ pivot).
- **S2:** Contains elements greater than the pivot (> pivot).

After partitioning, Quick Sort applies the same process recursively to (S1) and (S2) until the entire array is sorted.

Example: For the array [8, 3, 6, 2, 9, 4, 5, 1, 7], the pivot is 8 (the median of 8, 9, and 7). After partitioning, the array becomes [7, 3, 6, 2, 4, 5, 1, 8, 9], where the pivot is placed in its final position (index 7). The same process is then applied recursively to the elements on its left and right.

4. Pseudocode

```
QuickSort(A, left, right)
    if left < right
        middle = (left + right) / 2
        pivotIndex = MedianOfThreeIndex(
                        A, left, middle, right)
        swap A[pivotIndex] with A[right]
        p = Partition(A, left, right)
        QuickSort(A, left, p - 1)
        QuickSort(A, p + 1, right)

Partition(A, left, right)
    pivot = A[right]
    i = left - 1
    for j = left to right - 1
        if A[j] <= pivot
            i = i + 1
            swap A[i] with A[j]
    swap A[i + 1] with A[right]
    return i + 1
```

```
MedianOfThreeIndex(A, left, middle, right)
    if (A[left] <= A[middle] AND
        A[middle] <= A[right]) OR
       (A[right] <= A[middle] AND
        A[middle] <= A[left])
        return middle
    else if (A[middle] <= A[left] AND
             A[left] <= A[right]) OR
            (A[right] <= A[left] AND
             A[left] <= A[middle])
        return left
    else
        return right

Swap(A, i, j)
    temp = A[i]
    A[i] = A[j]
    A[j] = temp
```

5. Java Implementation and Output

5.1 Java Code

```
import java.util.Arrays;
import java.util.Random;
public class QuickSort {
    public static void quickSort(int[] arr, int left, int right) {
        if (left < right) {
            int middle = (left + right) / 2;
            int pivotIndex = medianOfThreeIndex(arr, left, middle, right);
            swap(arr, pivotIndex, right);
            int p = partition(arr, left, right);
            quickSort(arr, left, p - 1);
            quickSort(arr, p + 1, right);
        }
    }
    public static int partition(int[] arr, int left, int right) {
        int pivot = arr[right];
        int i = left - 1;
        for (int j = left; j < right; j++) {
            if (arr[j] <= pivot) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i + 1, right);
        return i + 1;
    }
    public static int medianOfThreeIndex(int[] arr, int left, int middle, int right) {
        if ((arr[left] <= arr[middle] && arr[middle] <= arr[right]) ||
            (arr[right] <= arr[middle] && arr[middle] <= arr[left])) {
            return middle;
        }
        else if ((arr[middle] <= arr[left] && arr[left] <= arr[right]) ||
                 (arr[right] <= arr[left] && arr[left] <= arr[middle])) {
            return left;
        }
        else {
            return right;
        }
    }
    public static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
    public static boolean isSorted(int[] a) {
        for (int i = 1; i < a.length; i++)
            if (a[i - 1] > a[i]) return false;
        return true;
    }
```

```
    public static void main(String[] args) {
        // Part 1: Test sorting correctness
        int[][] tests = {
            {8, 3, 6, 2, 9, 4, 5, 1, 7},   // random
            {1, 2, 3, 4, 5},               // already sorted
            {5, 4, 3, 2, 1},               // reverse
            {4, 2, 4, 2, 4},               // duplicates
            {7},                           // single element
            {}                             // empty
        };
        System.out.println("Correctness Tests:");
        for (int[] t : tests) {
            int[] copy = t.clone();
            quickSort(copy, 0, copy.length - 1);
            int[] expected = t.clone();
            Arrays.sort(expected);
            System.out.println(Arrays.toString(t) + " -> " + Arrays.toString(copy)
                + (Arrays.equals(copy, expected) ? "  PASS" : "  FAIL"));
        }
        System.out.println();
        // Part 2: Performance Testing
        int[] sizes = {100, 1000, 10000, 100000};
        Random random = new Random();
        System.out.println("Quick Sort Performance:");
        System.out.println("Size\tAverage Time (ms)");
        for (int size : sizes) {
            int[] original = new int[size];
            for (int i = 0; i < size; i++) {
                original[i] = random.nextInt(100000);
            }
            for (int w = 0; w < 10; w++) {
                int[] warmUp = Arrays.copyOf(original, size);
                quickSort(warmUp, 0, warmUp.length - 1);
            }
            long totalTime = 0;
            for (int trial = 0; trial < 5; trial++) {
                int[] testArr = Arrays.copyOf(original, size);
                long startTime = System.nanoTime();
                quickSort(testArr, 0, testArr.length - 1);
                long endTime = System.nanoTime();
                totalTime += (endTime - startTime);
                if (!isSorted(testArr)) System.out.println("ERROR: not sorted!");
            }
            double averageTime = (totalTime / 5.0) / 1_000_000.0;
            System.out.printf("%d\t%.4f%n", size, averageTime);
        }
    }
}
```

5.2 Program Output

```
Correctness Tests:
[8, 3, 6, 2, 9, 4, 5, 1, 7] -> [1, 2, 3, 4, 5, 6, 7, 8, 9]  PASS
[1, 2, 3, 4, 5] -> [1, 2, 3, 4, 5]  PASS
[5, 4, 3, 2, 1] -> [1, 2, 3, 4, 5]  PASS
[4, 2, 4, 2, 4] -> [2, 2, 4, 4, 4]  PASS
[7] -> [7]  PASS
[] -> []  PASS

Quick Sort Performance:
Size    Average Time (ms)
100    0.0276
1000    0.0734
10000    0.5437
100000  11.2906
```

Figure 1: Output of the program.

Array Size (n)	Average Time (ms)
100	0.0276
1,000	0.0734
10,000	0.5437
100,000	11.2906

Table 1: Average running time on random arrays (average of 5 runs).

Each time in the table is the average of 5 timed runs, because a single measurement can be affected by factors unrelated to the algorithm, such as JVM activity and system load. Averaging reduces this effect and gives a more reliable estimate.

6. Time Complexity

The time complexity of Quick Sort depends on how the pivot divides the array into two partitions.

- **Best Case:** O(n log n) – Occurs when the pivot divides the array into two nearly equal partitions.
- **Average Case:** O(n log n) – Occurs when the partitions are reasonably balanced.
- **Worst Case:** O(n^2) – Occurs when the pivot repeatedly creates highly unbalanced partitions, such as one empty partition and one containing the remaining elements.

Using the Median-of-Three pivot selection method helps improve pivot selection and reduce the likelihood of highly unbalanced partitions. However, it does not eliminate the worst-case scenario.

Note that when all elements of the array are equal, our partition places all of them in S1, which leads to the worst case O(n^2).

7. Space Complexity

The space complexity of Quick Sort depends on the recursion depth.

- **Best Case:** O(log n) – Occurs when the pivot divides the array into two nearly equal partitions.
- **Average Case:** O(log n) – Occurs when the partitions are reasonably balanced.
- **Worst Case:** O(n) – Occurs when the pivot creates highly unbalanced partitions, resulting in deeper recursive calls.

Quick Sort uses in-place partitioning, which means it does not require an additional array to store the sorted elements. However, it still requires extra memory for the recursion call stack.

Effect of the pivot choice: A pivot close to the median divides the array into nearly equal partitions, which keeps the recursion shallow and gives O(log n) space and O(n log n) time. A poor pivot, such as the smallest or largest element, creates highly unbalanced partitions, which makes the recursion deeper and increases the space to O(n) and the time to O(n^2). This is why the Median-of-Three method is used.

8. Conclusion

In conclusion, this project introduced Quick Sort and its Divide-and-Conquer approach. We explained its working process, presented its pseudocode, and implemented it using Java. The algorithm was tested successfully, sorting the elements in ascending order. Quick Sort has an average time complexity of O(n log n), making it an efficient sorting algorithm.